FaceLit - implementación y despliegue en Hostinger

Actualización: 4 de noviembre de 2026. Estado del informe: documenta la publicación desde `main` y el procedimiento operativo. Al 4 de noviembre, el equipo confirma que login, CSV, correo, cámara y asistencia funcionan en el sistema.

Topología desplegada reportada

- Dominio web: https://facelit.tech/

- VPS de Hostinger con Docker Compose.

- Caddy sirve la exportación web y hace proxy HTTPS/WebSocket al backend.

- PostgreSQL 16 con pgvector, backend Spring Boot, y microservicio FastAPI están en red privada Docker.

- Solo el servicio web publica 80/443; PostgreSQL, backend y servicio facial no publican sus puertos al exterior.

- Servicio facial privado; backend y facial comparten la base. Liquibase actualiza el esquema antes de arrancar la nueva API.

Ramas y revisiones fuente

El flujo de publicación toma los cuatro repositorios desde `main`. Los SHA siguientes corresponden a los `main` locales consultados para actualizar este documento; el manifiesto generado por `deploy/package.ps1` debe conservar los SHA exactos que se empaquetan para cada release.

Repositorio	Rama	SHA completo
FaceLit	main	6efd92cdd7e4e358b131cbe860cd8585d13e5550
FaceLit-Backend	main	2ba739853a808c8e5902f70eae98706a612f91e6
micro-facial	main	166e4a7f2245456277d57015259c678d8febfa9f
FaceLit-DB	main	cd069ae0cb4947d4b7ebbff17e9d6dc280ab86fd

Estos SHA identifican el código fuente consultado, no demuestran por sí solos qué imágenes están activas en el VPS. El manifiesto del paquete y los digests de las imágenes usados en cada release son el registro de publicación que se debe conservar.

Procedimiento de despliegue desde `main`

El paquete se prepara desde los checkouts locales de los cuatro repositorios; no se combinan ramas. Los siguientes pasos describen el flujo aplicado para publicar una versión:

1. En cada repositorio (`FaceLit`, `FaceLit-Backend`, `micro-facial` y `FaceLit-DB`), cambiar a `main`, actualizar desde `origin/main` y comprobar que no haya cambios locales que vayan a entrar accidentalmente en la publicación.

2. Registrar los SHA de `main` seleccionados. Ejecutar `deploy/package.ps1` desde la raíz del proyecto. El script genera `deploy/release-manifest.json` con rama, SHA y estado de cambios locales, y crea un archivo `facelit-vps-<fecha>.tar.gz` con los componentes necesarios; excluye secretos, dependencias locales y bases de datos.

3. Revisar el manifiesto y el contenido del paquete. Detener el proceso si algún repositorio no indica `main`, si hay cambios locales no aprobados o si el SHA no es el que se autorizó para el release.

4. Crear un respaldo de PostgreSQL y verificar que pueda leerse. Conservar también los digests de las imágenes actualmente activas para poder volver a la versión anterior.

5. Transferir el paquete al VPS por SSH/SCP, extraerlo en un directorio de release nuevo y conservar intacto el directorio de la versión anterior.

6. Configurar allí el archivo privado `.env` con dominio, base de datos, JWT, clave interna del servicio facial y SMTP. No copiar ese archivo al paquete ni al repositorio.

7. Validar la configuración con `docker compose config --quiet`. Construir las imágenes de backend, servicio facial, web y migrador desde el paquete identificado por el manifiesto.

8. Iniciar PostgreSQL y ejecutar `migrate validate`. Revisar el resultado y aplicar las migraciones pendientes con `migrate update`; no borrar volúmenes ni reinicializar la base.

9. Levantar servicio facial, backend y web. Comprobar `docker compose ps`, los healthchecks y los logs de arranque. Confirmar que Liquibase terminó y que el backend puede conectarse a PostgreSQL y al servicio facial.

10. Verificar el dominio por HTTPS, la carga y recarga de rutas web, login, autorización, API y WebSocket. Probar los flujos funcionales acordados, incluyendo cámara/reconocimiento y correo SMTP, y registrar sus resultados.

11. Guardar manifiesto, SHA, digests, respaldo y resultados de comprobación junto al registro del release. Declarar la publicación aceptada solo cuando terminen las pruebas manuales previstas.

Registro de la actualización del 27 de octubre

El informe operativo de esa fecha registra respaldo PostgreSQL, validación del changelog, detención temporal de web/backend/servicio facial, actualización Liquibase de 39 a 45 changesets, limpieza transaccional autorizada que preservó la cuenta coordinadora y activación de contenedores Docker. También reporta comprobaciones HTTPS, API, autorización sin login y WebSocket. El manifiesto archivado de ese release indica `dev` para FaceLit, FaceLit-Backend y micro-facial, y `main` para FaceLit-DB; por eso ese registro histórico no debe presentarse como despliegue desde `main`. Los releases posteriores deben seguir el procedimiento de esta sección y conservar su propio manifiesto.

Los respaldos contienen datos privados y permanecen en el VPS con acceso restringido. No incluir rutas de acceso, credenciales ni respaldos en el paquete público de documentación.

Resultado de preparación de datos

Después de la limpieza transaccional reportada: 1 usuario coordinador, 1 credencial, 3 roles y 12 permisos; cero instructores, aprendices, programas, fichas, referencias faciales y eventos. La demostración requiere cargar antes el CSV de prueba y preparar instructor/aprendiz y sesión.

Comprobaciones técnicas registradas

- Sitio y ruta de login responden HTTPS 200; archivos JavaScript accesibles.

- Backend inicia con validación JPA del esquema.

- PostgreSQL y servicio facial saludables; endpoint de salud facial 200.

- API administrativa sin autenticación responde 401.

- API de aceptación legal se consultó directa y a través del proxy.

- Clave y URL interna entre backend y facial coinciden, sin imprimir sus valores.

- WebSocket HTTPS devuelve handshake 101 para el origen permitido.

- Base en 45 changesets, sin pendientes.

- DNS no se modificó durante esta activación.

Las comprobaciones técnicas enumeradas corresponden al registro anterior del despliegue. Al 4 de noviembre, el equipo confirma que el login real, la carga CSV, el correo SMTP, la cámara y la asistencia funcionan. Esta actualización no agrega una medición de precisión biométrica ni un ensayo de restauración.

Procedimiento operativo para actualización futura

1. Confirmar SHA de los cuatro repositorios y revisar cambios/migraciones.

2. Crear backup y verificar su integridad antes de cambios.

3. Crear una carpeta de release nueva; conservar el release anterior.

4. Validar Docker Compose y construir las imágenes identificadas por los SHA aprobados.

5. Detener temporalmente los consumidores si la migración lo exige.

6. Validar Liquibase, inspeccionar SQL previsto y aplicar migraciones compatibles.

7. Activar servicios, revisar healthchecks y logs, comprobar HTTPS/API/WebSocket.

8. En cada release, comprobar los flujos funcionales con coordinador de prueba, CSV controlado, cámara y SMTP, y guardar sus resultados.

9. Documentar SHA, resultados, evidencia, limitaciones y decisión de aceptación.

10. Si falla una migración, mantener la web detenida y seguir el plan de recuperación; volver a una imagen anterior no revierte automáticamente PostgreSQL.

No ejecutar una eliminación de volúmenes de Docker como operación de actualización. La reversión debe considerar compatibilidad de código y esquema, no solo contenedores.

Diferencia con el despliegue histórico

Los manuales previos describían Render/Vercel y un microservicio facial no desplegado. Esa secuencia ya no describe esta topología. La actualización del 27 de octubre es un antecedente de ejecución y su manifiesto registra ramas `dev` para tres repositorios; el flujo vigente documentado aquí prepara los releases desde `main`.

Referencias

Informe DESPLIEGUE_HOSTINGER_2026-10-27.md, deploy/compose.yaml, deploy/Caddyfile, deploy/package.ps1 y cambios de FaceLit-DB en 06_migrations/. El informe documenta el proceso; esta actualización no repite despliegue ni toca producción.

Criterios previos a cada publicación

Antes de desplegar cambios de registro, correo o consentimiento, confirmar que todas las migraciones estén enlazadas al changelog, que SMTP esté configurado y probado y que los controles de acceso hayan pasado pruebas. La presencia de código en `main` por sí sola no sustituye estas comprobaciones ni la evidencia del SHA del release.